Reviewer Pack — Automorphic L-Function Rank and Polynomial Hierarchy Depth

Entry c61d7d1f2a35 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 12:38:51 UTC

Automorphic L-Function Rank and Polynomial Hierarchy Depth

Entry ID: `c61d7d1f2a35` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 12:38:51 UTC

1. Conjecture

Field A (mathematical branch): Automorphic L-Functions **Field B** (complexity object): Polynomial Hierarchy Depth

Statement:

> {'complexity': 'For all Boolean formulas ϕ, the rank of the corresponding automorphic L-function L(ϕ, s) is at most Φ(n), where Φ is a polynomial function with Φ(n) ≤ log^k n for some constant k.', 'automorphic': 'The rank of an automorphic L-function L(ϕ, s) over Q with respect to ϕ is at most Φ(n), where Φ(n) = O(log^k n).'}

Rationale (proposer's reasoning):

> {'connection': 'Automorphic L-functions encode arithmetic structure in number theory, while polynomial hierarchy depth quantifies the complexity of computational problems. The conjecture suggests a potential bridge between these fields by proposing a bound on the rank of automorphic L-functions related to the size of their corresponding Boolean formulas.', 'relevance': 'If true, this would provide a new way to understand the structure of the polynomial hierarchy and could lead to insights into complexity classes that are currently difficult to analyze.'}

Taxonomy category: `PROOF_COMPLEXITY_EXT_FREGE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ff5d363e0089feb8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For all generated Boolean formulas ϕ with size n up to 40, if the rank of the automorphic L-function L(ϕ, s) is less than or equal to Φ(n), where Φ(n) ≤ log^k n and k is a constant, then the conjecture is supported. Falsification occurs if any seed produces an instance with a rank of L(ϕ, s) greater than Φ(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphic L-function AND polynomial hierarchy depth - L-function rank Automorphic AND complexity of Boolean formulas - polynomial Hierarchy Depth in relation to Automorphic L-Function rank

Top relevant hits considered: - [http://arxiv.org/abs/0909.4925v3] Higher regularizations for zeros of cuspidal automorphic L-functions of GL_d - [http://arxiv.org/abs/1604.04253v2] On Period Relations for Automorphic L-functions II - [http://arxiv.org/abs/2512.22451v1] Zeros of Polynomials in Derivatives of Automorphic L-functions - [http://arxiv.org/abs/1801.03881v2] Relative functoriality and functional equations via trace formulas - [http://arxiv.org/abs/2311.01864v1] SortNet: Learning To Rank By a Neural-Based Sorting Algorithm - [http://arxiv.org/abs/1311.6168v2] p-adic L-functions of automorphic forms and exceptional zeros - [http://arxiv.org/abs/1710.06598v2] A Bounded Degree Lasserre Hierarchy with SOCP Relaxations for Global Polynomial Optimization and Applications

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=198.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        if n == 1:
            return '0' if random.choice([True, False]) else '1'
        elif n == 2:
            return f"({'0' if random.choice([True, False]) else '1'}) {'AND' if random.choice([Tru
        else:
            subformulas = [generate_boolean_formula(random.randint(1, n-1)) for _ in range(n-1)]
            return f"({' AND '.join(subformulas)}) {'AND' if random.choice([True, False]) else 'OR

    def compute_l_function_rank(formula):
        # Placeholder function to simulate L-function rank computation
        # In practice, this would involve complex number theory and automorphic forms
        return len(formula.split())
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per size
            formula = generate_boolean_formula(n)
            rank = compute_l_function_rank(formula)
            Phi_n = math.log(n) ** 2  # Example polynomial function Φ(n) = log^2(n)
            results.append({
                "n": n,
                "formula": formula,
                "rank": rank,
                "Phi_n": Phi_n
            })

    max_rank = max(result["rank"] for result in results)
    conjecture_holds = all(max_rank <= result["Phi_n"] for result in results)
    counterexample = "" if conjecture_holds else f"Formula: {results[max(results, key=lambda x: x[

    return {
        "metric_name": "L-function rank",
        "metric_value": max_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{results[max(results, key=lambda x: x['rank'])]
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means we cannot verify if all seeds produced L_function_rank <= Φ(n). | next: Re-run the test with increased timeout limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16579
2	preregistration	ollama_remote	glm4:latest	0	0	16084
3	novelty	ollama_remote	glm4:latest	0	0	9444
4	novelty	ollama_remote	glm4:latest	0	0	10234
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12390
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9747
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8243
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9892
9	judge	ollama_remote	glm4:latest	0	0	90671

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 183283 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c61d7d1f2a35.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c61d7d1f2a35.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c61d7d1f2a35.tar.gz` (if generated)